Freight Rate Prediction

Machine Learning Engineer assessment: validation approach, results and December forecast

1.60%	$38	+0.09%	1.56%
MAPE, 6-fold time-based backtest (2-month horizon)	mean absolute error per load (R^2 0.998)	average bias across the six folds	MAPE on the Sep–Oct holdout (closest to the real task)

Summary

- **The task is a forecast.** The development data covers Jan 1 – Oct 31 2025; the 12,000 loads to predict are the next two months (Nov–Dec 2025). Every validation fold therefore trains on the past and tests on the following two months; no shuffled splits are used for model selection.
- **1.41% of labels are corrupted** (677 rows priced 2.2–5.4x too high or 0.17–0.45x too low). They are removed from training and reported separately in evaluation.
- **Final model:** *log(rate) = log(distance) + load effect + market effect*. LightGBM learns what makes one load cost more than another on the same day; a linear model learns what moves every price on a given day (market index, a slow trend and a quarter-end ramp) and can extrapolate to Nov–Dec.
- **The split choice decides the model.** On a random 5-fold split a single LightGBM looks best (0.99% MAPE); forward in time it scores 4.70%, three times worse than the final model.
- **December chart:** the fixed Lexington → Fort Wayne load rises from $827 (Dec 1) to $890 (Dec 31): a weekly cycle that peaks on Thursdays, on top of the quarter-end ramp.

1. Data and data-quality handling

train_test.csv holds 48,000 labelled loads (Jan–Oct 2025, 64 cities). **validation.csv** holds 12,000 unlabelled loads (Nov–Dec 2025, 72 cities). Each load has pickup and delivery city with coordinates, distance, equipment (Dry Van, Reefer, Flatbed), weight, date, *market_index* and *quote_signal*; the target is *posted_rate*.

Issue	Extent	Evidence	Handling
Corrupted rate labels	677 rows (1.41%)	Two separate clusters at 2.2–5.4x and 0.17–0.45x a robust baseline, with an empty gap to the clean rows (0.90–1.11x). Spread evenly over months, lanes and equipment.	Removed from training; excluded from headline metrics, reported in an all-rows MAE
Negative weight	292 train / 145 val	Absolute values match the positive distribution (sign flips).	abs(weight)
Weight clipped	1,204 at 47,500 lb; 32 at 5,000 lb	Spikes at both ends of a smooth distribution.	Kept, plus an at-cap flag
Missing values	weight 0.6%, market_index 0.8%	Random across dates.	Weight: NaN + flag (LightGBM). market_index: that day's mean
Unseen cities	8 cities, 12.1% of validation rows	Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk, San Diego never appear in training.	Location modelled via coordinates, never city IDs

Issue	Extent	Evidence	Handling
Coordinates are not real geography	All cities	e.g. Boston at lat 39.3, but each city has one fixed pair and distance = 1.18x the straight-line distance (corr 0.9995).	Used as consistent coordinates

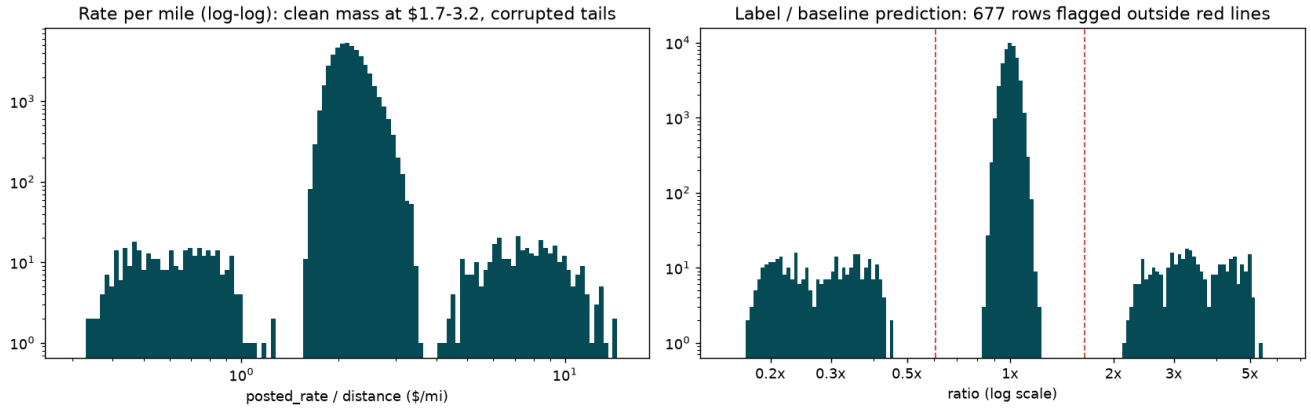


Figure 1. Left: rate per mile, with clean rows in the centre and the corrupted tails either side. Right: label divided by a robust baseline prediction; the red lines at 0.61x and 1.65x sit in empty gaps, so the flagging is not sensitive to the exact threshold.

2. Train / test split and validation approach

2.1 Why the split must follow time

The loads to predict lie entirely after the development data. Prices share strong date-level components: *market_index* has a weekly cycle and a seasonal level, rates drift upward over the year, and they ramp up through the last month of every quarter (Section 4). In a shuffled split, every test day has training rows from the same day, so a model can memorise each day's price level, which it will never have for Nov–Dec. The validation scheme therefore reproduces the real situation: **train on the past, test on the next two months**.

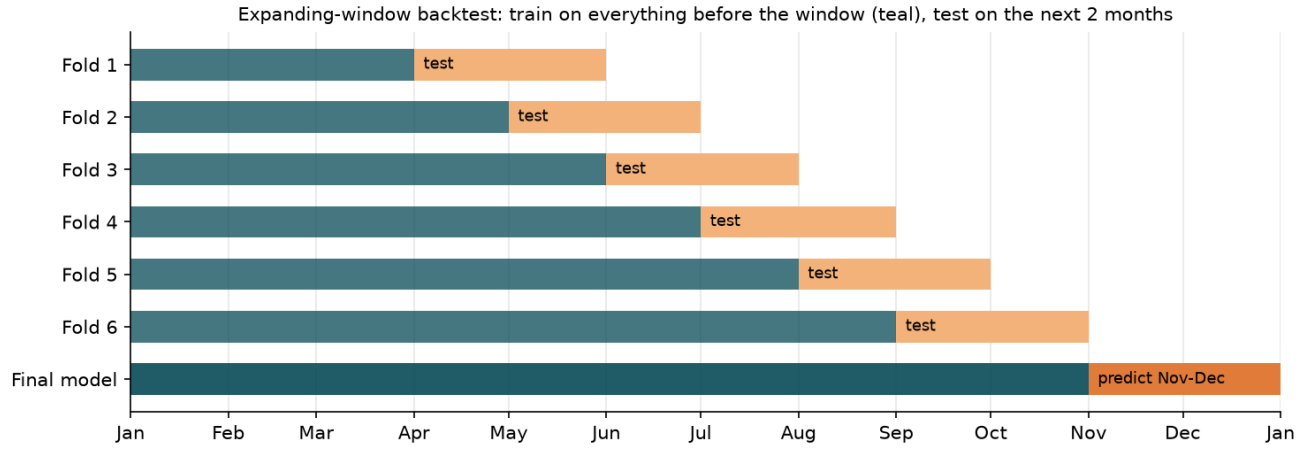


Figure 2. Six expanding-window folds with a 2-month test window, moving forward one month at a time. The final model is trained on all of Jan–Oct and predicts Nov–Dec.

2.2 Validation design

Step	What was done	Why
Label QA	Corrupted labels flagged once on the development set: rows more than exp(±0.5) from a robust log-linear baseline (distance, equipment, daily market index).	Not real prices, so no model can or should predict them. Re-flagging inside each fold gives exactly the same rows (0 disagreements in all 6 folds), so flagging once leaks nothing.
Training folds	Flagged rows dropped from every training fold.	Stops 3–5x labels from distorting the fit.
Main scheme	Expanding-window backtest, 6 folds, 2-month test windows (Apr–May … Sep–Oct).	Same horizon as the real task. Covers every season; June and September, both quarter-end months, fall inside test windows.
Primary fold	Train Jan–Aug, test Sep–Oct.	The closest stand-in for train Jan–Oct, predict Nov–Dec.
Contrast	Random 5-fold CV on the same data.	Shows how optimistic a shuffled split is.
Unseen cities	8 random cities removed from the primary fold's training data; test rows touching them scored separately.	Mirrors the 8 validation cities that never appear in training.
Metrics	MAPE, MAE, RMSE, bias (mean log error) and R^2 on clean holdout rows, plus MAE on all holdout rows.	Corrupted labels add error no model can remove, so they would hide real differences.

Leakage checks. Daily market-index means are computed from the features of that day's loads only, never from labels. Model choices (feature set, market terms, hyper-parameters) were made on the backtest; the validation file was used only for its features (daily market index) and the final predictions.

3. Validation results

Mean over the six expanding-window folds (metrics on clean holdout rows; last column includes corrupted labels):

Model	MAPE %	MAE $	RMSE $	Bias %	R^2	MAE $ (all rows)
Lane × equipment median $/mile	5.01	116.5	166.8	−2.89	0.985	167.9
Single LightGBM, all features	4.70	99.6	145.7	−2.33	0.980	151.2
Log-linear (same market terms)	2.26	51.9	76.3	+0.04	0.997	104.2
Two-stage model (final)	**1.60**	**38.3**	**55.8**	**+0.09**	**0.998**	**90.7**

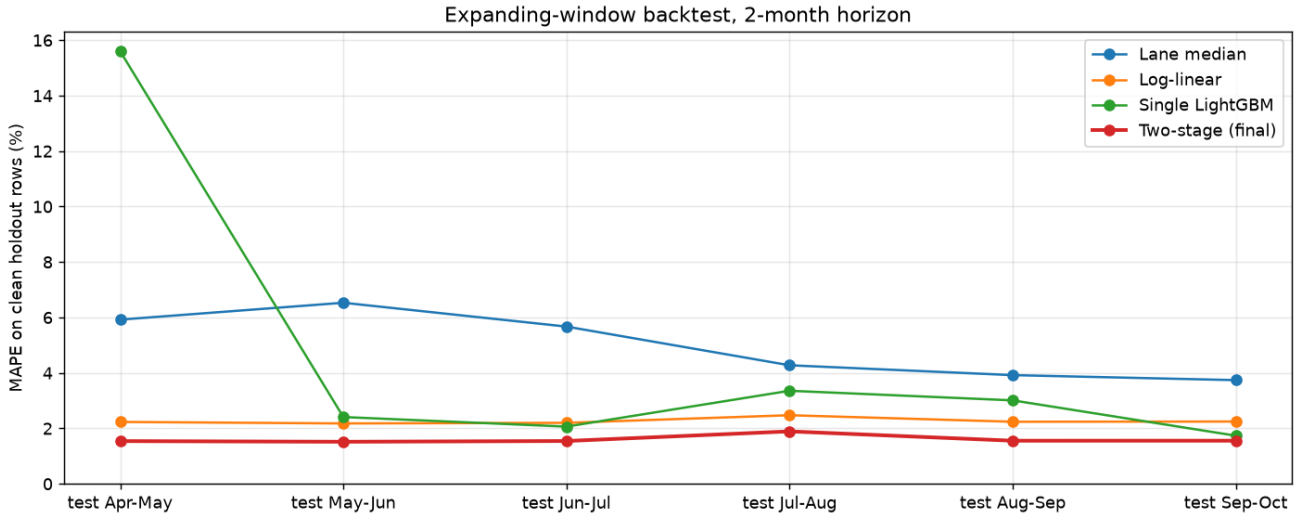


Figure 3. MAPE per fold. The final model is stable at 1.52–1.89% in every season. The single LightGBM reaches 15.6% on Apr–May. Most of that comes from quote_signal (3.3% without it); the rest comes from market-index levels above its Jan–Mar range (daily max 1.21 vs 1.40), which trees cannot extrapolate.

Shuffled vs time-based split

Model	Random 5-fold MAPE %	Time-based backtest MAPE %	Ranking
Single LightGBM	0.99	4.70	Best on random split, 3rd forward in time
Two-stage model (final)	1.43	1.60	Best forward in time

A random split would have selected the wrong model. Its 0.99% comes from memorising each day's price level from same-day training rows, information that does not exist when forecasting two months ahead.

Unseen cities and remaining error

- **Unseen cities:** rows touching 8 held-out cities score 1.72% MAPE (2,116 rows) vs 1.55% for seen cities (7,407 rows). The location effect is mostly a north–south gradient that coordinates capture.
- **Remaining error is mostly noise:** if each test day's market level were known exactly, MAPE would be about 1.40–1.50%. Forecasting the daily level costs only about 0.1–0.4 points.
- **Per fold (final model):** 1.54, 1.52, 1.55, 1.89, 1.56, 1.56% MAPE; bias between −0.61% and +1.39%; Sep–Oct MAE $36.2.

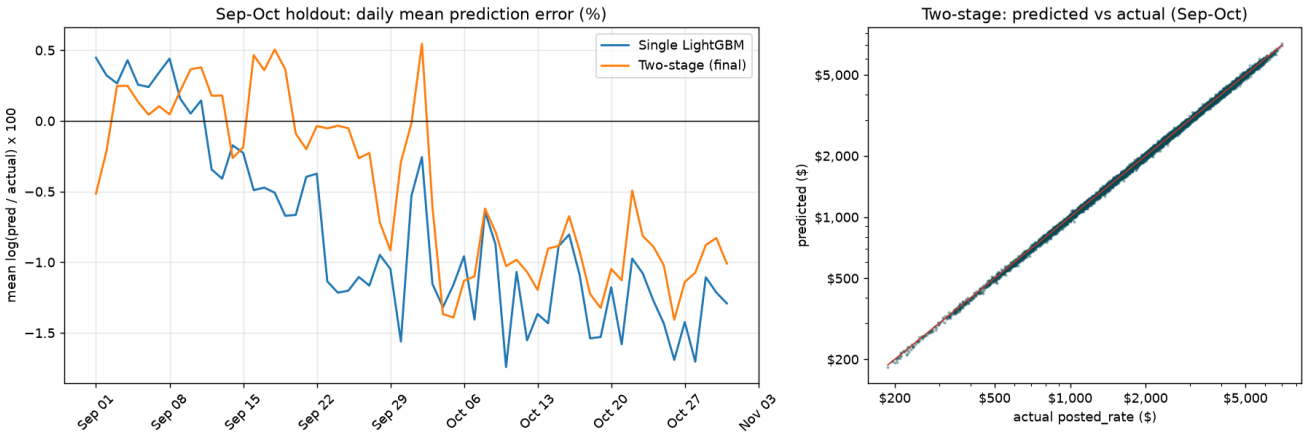


Figure 4. Primary Sep–Oct holdout. Left: daily mean error. The single LightGBM drifts to about −1.5% in October. The final model tracks September closely and runs about 1% low in October, when Q4 opened higher than the trend predicts. Right: predicted vs actual for the final model.

4. Model and why

Exploration showed the price is almost exactly multiplicative, with two different kinds of driver:

- **What makes one load cost more than another on the same day:** distance (non-linear: rate grows as distance$^{0.87}$), equipment (Reefer +13%, Flatbed +8% vs Dry Van), weight, and origin/destination (south/Texas about +3%, north-east about −3%). Stable over time and non-linear, which suits gradient-boosted trees.
- **What moves every price on a given day:** the daily *market_index* (weekly cycle peaking Thursday, seasonal level), a slow upward trend (about +0.6% per month) and a quarter-end ramp (flat in months 1–2 of each quarter, rising about 4% through month 3, then resetting). These must be extrapolated to dates and index levels outside the training data, which trees cannot do.



Figure 5. Top: daily market index, weekly cycle on a seasonal level, continuing into the validation period. Bottom: daily rate residual after the market index is removed. Rates build through each quarter and reset on Apr 1, Jul 1 and Oct 1.

Model structure

```
log(rate) = log(distance) + load_effect(x) + market_effect(date, market_index)
```

Component	Model	Inputs
Load effect	LightGBM on log rate per mile	log distance, equipment, weight (+ missing and at-cap flags), pickup and delivery lat/lon
Market effect	Linear regression	market_index, trend (days since Jan 1), quarter-end-month indicator, days into the quarter-end month

The two parts are fitted by **backfitting**: the trees are trained against per-date fixed effects (three passes), so day-level movements never leak into the load model; the linear market model is then fitted on what remains. Final coefficients: market_index 0.141 (0.116–0.145 across folds), trend +0.0197% per day, quarter-end ramp +0.135% per day. LightGBM settings: 31 leaves, learning rate 0.03, 1,200 rounds, 0.8 row / 0.9 feature subsampling. 31–127 leaves and squared-error vs Huber loss all landed within 1.60–1.65% mean MAPE.

Design decisions tested on the same backtest

Variant (mean over 6 folds)	MAPE %	Decision
Final model	**1.60**	**Selected**
Market model with market_index only	2.80	Trend and ramp terms needed
market_index + trend, no quarter-end ramp	2.46	Ramp needed; without it the trend absorbs quarter ends
+ day-of-week dummies	1.59	No real gain; market_index already carries the weekly cycle
+ quote_signal in the market model	1.61	No gain
+ quote_signal deviation in the trees	4.19	Rejected: 1.28% on Sep–Oct but 14.48% on Apr–May
+ market_index within-day deviation in the trees	1.70	Rejected; the linear term already covers it
No backfitting	1.83	Backfitting matters

Why quote_signal is excluded. Its correlation with price changes sign between periods: +0.30 to +0.41 in the second half of Mar, Jun and Sep, about 0 from April to August, and −0.14 in October. Nov–Dec resemble August, when it carried no signal. A feature whose sign cannot be predicted is a liability two months ahead.

5. December prediction chart

The chart input holds one load (Lexington → Fort Wayne, 360 miles, Dry Van, 32,000 lb) for each day of December 2025. It has no coordinates, *market_index* or *quote_signal*. Coordinates come from the city lookup. *market_index* for each date is that day's mean over the December loads in validation.csv (about 200 loads per day). This is a feature, not a label, so no information leaks.

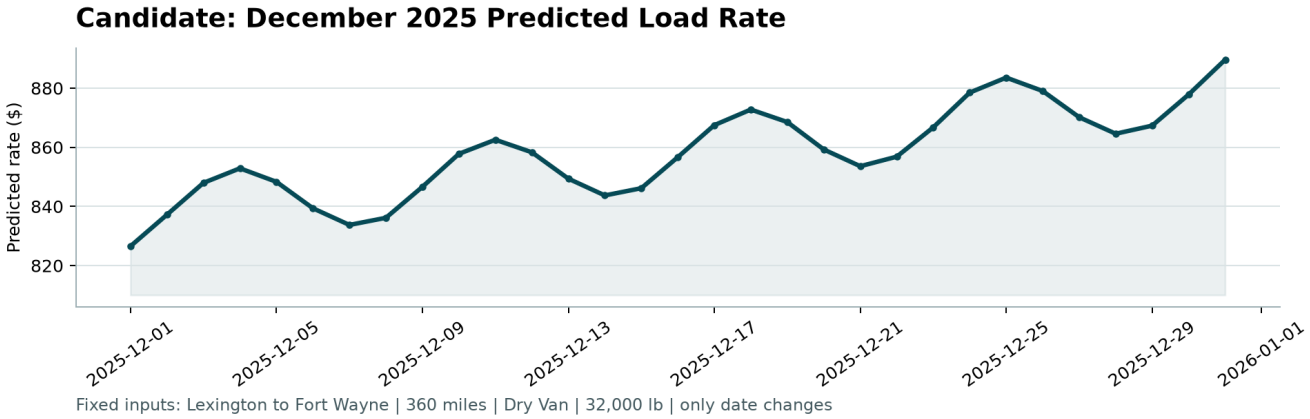


Figure 6. December chart produced by the provided score.py from data/december_chart_inputs.csv.

The forecast rises from **$827 on Dec 1 to $890 on Dec 31**. Three components drive the shape:

- **Weekly cycle (about ±1.5%)**: the December market index peaks on Thursdays and bottoms on Sundays/Mondays, continuing the pattern seen throughout training.
- **Quarter-end ramp (+4.1% over the month)**: December is the third month of Q4, and rates climbed the same way through March, June and September.
- **Trend (+0.6% over the month)**: the slow upward drift seen across 2025.

For reference, Dry Van loads on this lane in training cost $758–$935. Across all 12,000 validation loads, the predicted Dry Van median rate per mile follows on from October ($2.07 actual) to $2.05 in November (lower market index) and $2.11 in December (quarter-end ramp).

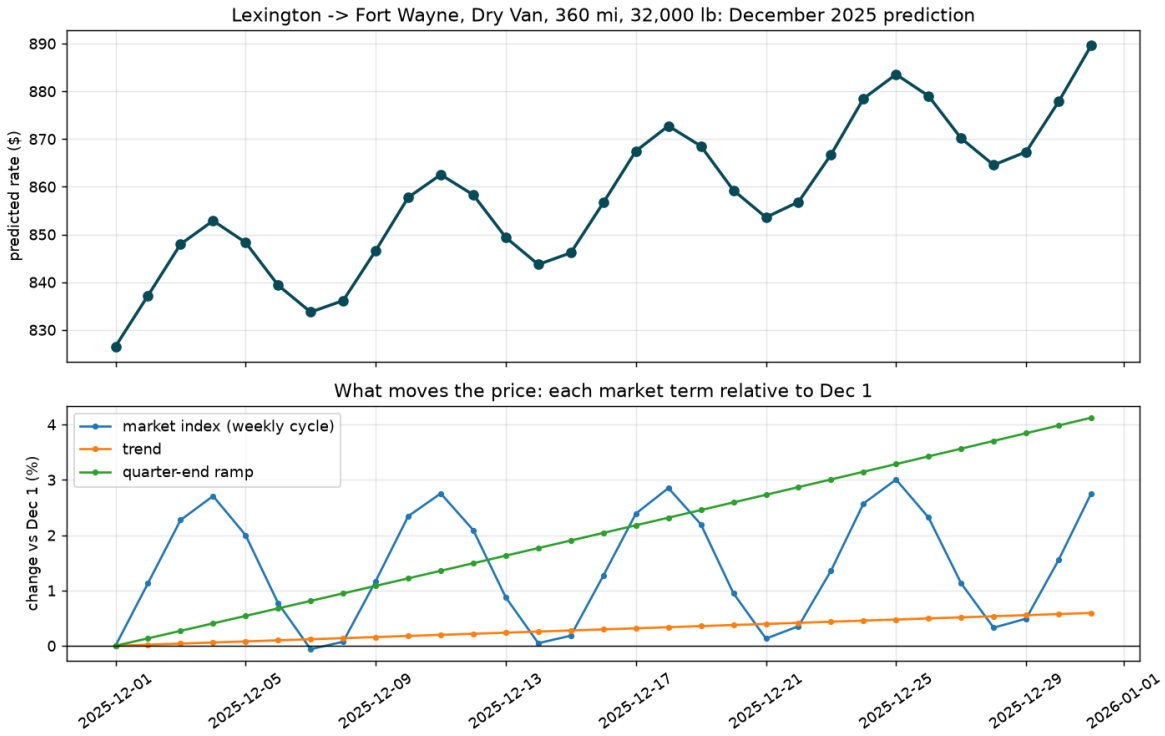


Figure 7. Top: December prediction. Bottom: each market term's contribution relative to Dec 1.

Appendix: reproducing the results

Command	What it does
`python -m pip install -r requirements.txt`	Install dependencies
`python src/train.py`	Fit on Jan–Oct; write validation_predictions.csv and fill data/december_chart_inputs.csv
`python score.py --predictions validation_predictions.csv --december-predictions data/december_chart_inputs.csv`	Provided scorer: validates both files, draws the December chart
`python src/validate.py`	Backtests, random-CV contrast, unseen-city check (figures 2–4)
`python src/ablation.py`	Design decisions table
`python src/eda.py`	Exploration statistics and figures

Code: *src/features.py* (loading, cleaning, label QA, features), *src/model.py* (two-stage model), *src/train.py* (final fit and outputs), *src/validate.py* and *src/ablation.py* (validation). Detailed notes: *reports/eda_findings.md* and *reports/modeling.md*.